

Contents of Synopsis

Topics	Page No.
Abstract	2
Synopsis	3
1. Name / Title of the Project	3
2. Statement About the Problem	3
3. Why is the Particular Topic Chosen?	4
4. Proposed Solution	4
5. Objective and Scope of the Project	5
6. Hardware and Software to be Used	6
7. Methodology (Summary of the Project)	7
8. USE CASE Diagram	9
9. Data Flow Diagram (DFD)	10
10. Entity Relationship Diagram (ERD)	12
11. What Contribution Would the Project Make?	13
References	14

EVENTTICKET

A Full-Stack Real-Time Event Ticket Booking Web Application

(Engineered with React 19, Express, WebSockets, JWT Authentication & Dynamic QR Ticketing)

Project Title	EventTicket — Real-Time Ticket Booking Platform
Technology Stack	React 19, TypeScript, Tailwind CSS, Express, Node.js
Real-Time Protocol	Socket.io (Bidirectional WebSocket Rooms)
Authentication	JSON Web Tokens (JWT) + bcryptjs Password Hashing
Digital Ticketing	Dynamic QR Code Generation (qrcode.react) + Digital Pass
Database Layer	Persistent Storage Engine / MongoDB Mongoose Compatible
Department & Year	Computer Science & Engineering (Diploma / B.Tech) 2025–2026

ABSTRACT

In the contemporary digital era, event organization across educational institutions, technical conferences, collegiate fests, and workshops demands instantaneous, reliable, and frictionless ticket reservation. Traditional ticketing portals are often burdened with mandatory third-party payment gateways, sluggish server response times, high commission overheads, and severe concurrency bottlenecks that frequently lead to overbooking and database race conditions when multiple users contest the same limited seats.

EventTicket is an advanced, production-grade full-stack web application designed specifically to eliminate payment friction through instant one-click reservations while preserving rigorous seat availability integrity via real-time WebSocket synchronization (Socket.io). Built on a modern decoupled architecture using React 19, TypeScript, Tailwind CSS, and Node.js Express, the platform delivers instantaneous multi-client seat decrements across all connected viewports without requiring manual browser refreshes.

The application features an interactive visual auditorium seating grid (Rows A–D, Seats 1–8), automated unique alphanumeric booking identifiers (TKT-YYYYMMDD-XXXX), dynamic high-density QR code digital passes, and an email-based "My Bookings" retrieval portal. For event coordinators, EventTicket incorporates a secure JWT-authenticated administrative dashboard featuring live KPI counters, complete event CRUD capabilities, and a 1-click QR/ID venue gate check-in scanner to mark tickets as "Used".

SYNOPSIS

1. Name / Title of the Project

Project Title: EventTicket — Full-Stack Real-Time Event Ticket Booking Web Application

EventTicket is a specialized, responsive real-time web application that facilitates rapid discovery, seat reservation, and digital ticketing for campus and community events. Utilizing event-driven WebSockets and stateless token-based authorization, the platform runs seamlessly across all modern web browsers without requiring third-party software installation or payment gateway credentials.

2. Statement About the Problem

Thorough investigation of existing commercial and open-source ticketing platforms reveals several critical shortcomings when applied to collegiate, community, and non-commercial environments:

- **Payment Gateway Roadblocks:** Most ticketing platforms mandate credit/debit card merchant integrations, adding friction, transaction charges, and privacy concerns for free campus symposiums and student club events.
- **Concurrency Collisions & Overbooking:** In high-demand event launches, conventional polling or standard HTTP request-response architectures fail to communicate instant seat reductions, resulting in multiple attendees booking the same final seats.
- **Lack of Real-Time Viewport Sync:** Users must repeatedly press manual refresh to ascertain if tickets remain available, causing unnecessary server traffic and degraded user experiences.
- **Vulnerable Paper & Static Tickets:** Static PDF attachments or paper receipts are easily replicated, modified, or forwarded, lacking live digital verification at entry gates.
- **Absence of Integrated Gate Validation:** Event coordinators lack a simple, unified, JWT-protected administrative dashboard to monitor live capacity and scan/validate tickets upon attendee arrival.

3. Why is the Particular Topic Chosen?

- **Practical Industry Relevance:** Combines mission-critical modern full-stack competencies: bidirectional WebSockets, asynchronous state management, JWT security, and atomic database updates.
- **Solves a Tangible Real-World Need:** Universities and technical societies frequently organize hackathons, guest lectures, cultural fests, and gaming championships that demand immediate registration without financial barriers.
- **Demonstrates Concurrency Mastery:** Illustrates how server-authoritative state models and WebSocket rooms prevent negative seat quantities during peak traffic spikes.
- **Comprehensive End-to-End Architecture:** Spans the complete lifecycle from attendee discovery, visual seat selection, and vector QR pass rendering to administrative gate check-ins.

4. Proposed Solution

EventTicket directly resolves the identified problems by providing a high-performance, real-time web portal engineered with the following robust modules:

- **Instant Zero-Payment Booking Engine:** Users enter only attendee name and email to immediately secure seats with no credit cards, merchant APIs, or payment timeouts required.
- **WebSocket Room Synchronization:** Built with Socket.io; whenever a reservation is confirmed, the server decrements seats atomically and broadcasts `seat-updated` to `event-\${id}` and global channels within milliseconds.
- **Visual Auditorium Seating Plan:** Features an interactive 4-row by 8-column seat layout (Rows A–D, Seats 1–8) with real-time occupied/reserved indicators coupled to a rapid ticket counter (1–6 seats).
- **Dynamic Digital Boarding Pass with QR Code:** Every successful booking automatically generates a unique booking ID (e.g., TKT-20260914-XXXX) and encodes verification metadata into an SVG QR code.
- **Print & PDF Optimization:** Equipped with specialized CSS print media queries allowing attendees to print or save crisp physical or PDF passes directly from any browser.
- **Email-Based "My Bookings" Self-Service:** Users can retrieve and reprint all historical and upcoming ticket passes simply by entering their email address.
- **Protected Admin Command Center:** JWT Bearer-token secured portal providing live KPI metrics, full event CRUD controls, and an instant gate check-in scanner to mark tickets as "Used".
- **Adaptive Dark/Light Interface:** Designed with Tailwind CSS supporting persistent theme preference toggling and responsive scaling across mobile, tablet, and desktop viewports.

5. Objective and Scope of the Project

Objectives:

1. To build a responsive, single-page full-stack ticketing platform using React 19, TypeScript, and Express.
2. To implement bidirectional real-time communication via Socket.io for instantaneous seat count synchronization.
3. To engineer a secure JWT-authenticated admin module using bcryptjs password hashing and route middleware.
4. To automate vector QR code generation (qrcode.react) containing embedded attendee verification payloads.
5. To develop a client-side lookup mechanism enabling attendees to search and retrieve booked passes via email.
6. To enforce server-side validation ensuring available seat counts never go negative under concurrent booking bursts.

Scope of the Project:

What is In Scope:

- Real-time multi-client seat updates for all listed events.
- Instant ticket reservations with attendee name and email.
- Interactive seating layout selection and quick quantity stepper.
- Unique booking ID and encrypted QR code pass rendering.
- Dedicated print stylesheets for physical boarding pass generation.
- JWT-protected admin dashboard with event CRUD and ticket check-in.

What is Out of Scope:

- Third-party commercial payment gateways (credit cards / Stripe / PayPal).
- Native compiled mobile apps (distributed as a universal responsive PWA-ready web application).
- Facial biometric attendee identification at gates.

Hardware Requirements:

Component	Minimum Requirement
Processor	Intel Core i3 / AMD Ryzen 3 or equivalent
System Memory (RAM)	4 GB RAM minimum (8 GB recommended for dev)
Storage Space	500 MB free hard disk space for Node environment
Internet Connection	Active broadband or 4G/5G mobile connection
Display Resolution	1024 x 768 pixels minimum (responsive mobile supported)
Input Devices	Standard Keyboard and Mouse / Smartphone Touchscreen

Software Requirements:

Category	Tool / Technology Used
Frontend Library	React 19 (Hooks, Functional Architecture)
Programming Language	TypeScript (Strict Static Typing)
Styling & Layout	Tailwind CSS v4 (Utility-first, Dark Mode Support)
Backend Server	Node.js runtime with Express.js Framework
Real-Time WebSockets	Socket.io (Engine.IO Server & Socket.io-Client)
Authentication	jsonwebtoken (JWT) + bcryptjs Password Hashing
QR Code Generation	qrcode.react (SVG / Canvas dynamic generator)
Database Engine	Atomic Persistent JSON Store / MongoDB Mongoose Compatible
Build & Bundling Tool	Vite 6 + esbuild + tsx execution engine
Development Environment	Visual Studio Code (VS Code)
Supported Browsers	Google Chrome, Mozilla Firefox, Microsoft Edge, Safari

7. Methodology (Summary of the Project)

The EventTicket system is developed using an agile, modular full-stack lifecycle divided into six systematic engineering phases:

Phase 1 — Requirements Elicitation & Domain Modeling:

- Identified bottlenecks in campus ticket distribution: manual queues, overbooking, and payment hurdles.
- Defined entity schemas for Events, Bookings, and Admin Credentials.
- Formulated real-time concurrency rules to ensure $\text{availableSeats} \geq \text{requestedSeats}$ before booking execution.

Phase 2 — Server Architecture & WebSocket Integration:

- Built Express HTTP server integrated with Socket.io on port 3000.
- Established room management (``join-event``, ``leave-event``, ``join-admin``) for localized event broadcast.
- Implemented atomic database operations ensuring zero race conditions during simultaneous bookings.

Phase 3 — JWT Authentication & Security Hardening:

- Configured bcryptjs password hashing with 10 salt rounds for administrator accounts.
- Implemented stateless JWT issuance with 7-day expiration and Authorization Bearer header extraction.
- Protected admin API endpoints (``/api/admin/*``) via dedicated Express middleware.

Phase 4 — Frontend UI & Interactive Seat Picker (React + Tailwind):

- Designed responsive homepage with dynamic category filters, search bar, and live availability pill badges.
- Created interactive 32-seat visual auditorium layout (Rows A–D, Seats 1–8) with rapid quantity counters.
- Engineered frictionless checkout form capturing name and email with instant validation.

Phase 5 — Digital Pass & QR Code Generation:

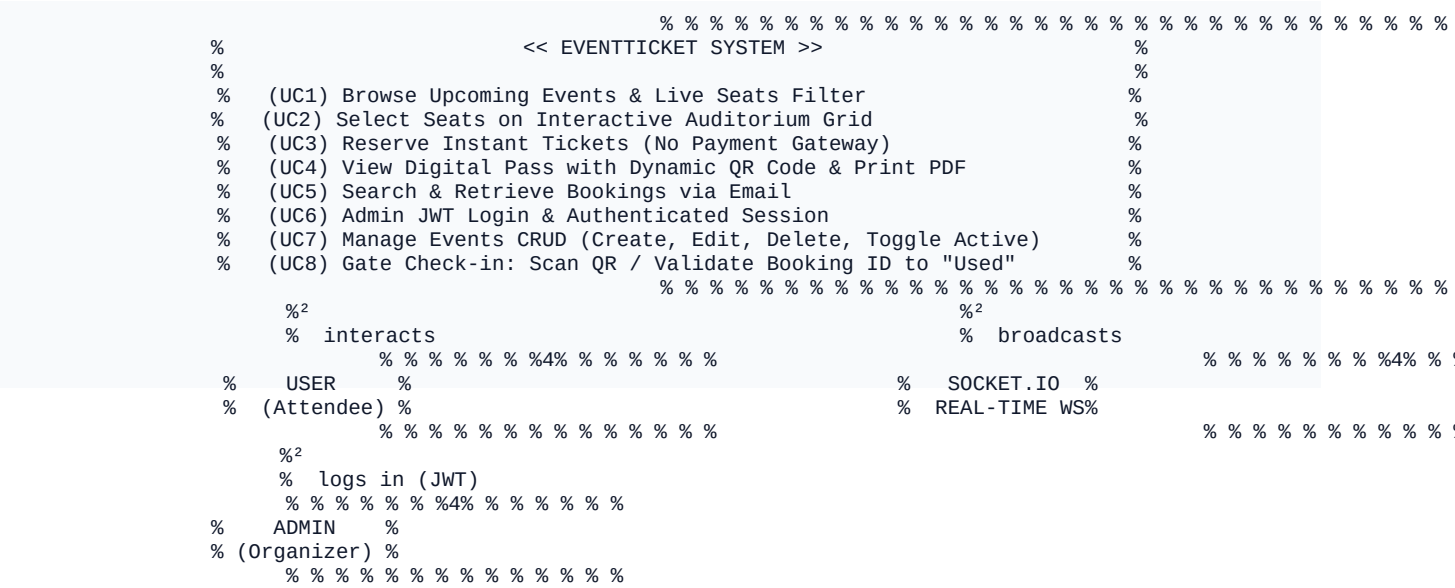
- Engineered unique booking identifier generator formatted as ``TKT-YYYYMMDD-XXXX``.
- Rendered high-density SVG QR codes storing verifiable cryptographic booking verification payloads.
- Crafted realistic perforated boarding pass UI with dedicated ``@media print`` stylesheets.

Phase 6 — Testing, Verification & Gate Validation Scanner:

- Conducted multi-browser WebSocket concurrency testing verifying sub-second seat updates across tabs.
- Verified JWT token expiration, invalid credential handling, and route protection.
- Built administrator check-in scanner validating booking IDs and marking passes as "Used".

8. USE CASE Diagram

The Use Case diagram illustrates the operational actors (Attendee / Browser User and Event Administrator) interacting with the EventTicket system boundary and the background WebSocket broadcast engine.



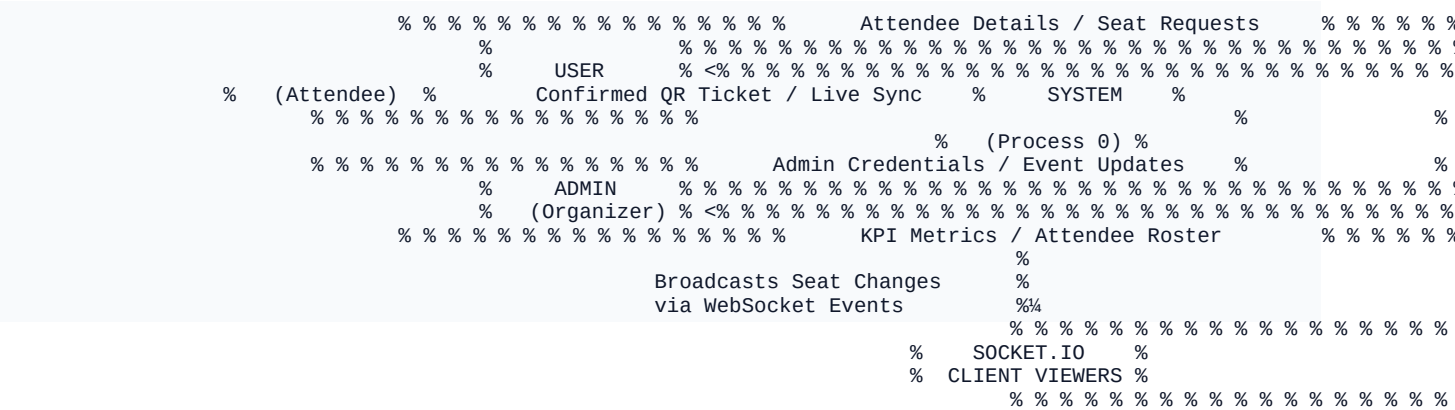
Use Case Descriptions:

Use Case ID & Name	Description
UC1: Browse Events	User views live event catalog with real-time available seat counter badges.
UC2: Select Seats	User selects 1–6 tickets or picks exact seats (Rows A–D) on visual layout.
UC3: Instant Booking	User inputs name and email; seats are atomically decremented on server.
UC4: View QR Pass	System generates unique TKT ID and SVG QR code ready for PDF download/print.
UC5: My Bookings	User enters email address to retrieve all associated active/past reservations.
UC6: Admin Auth	Admin logs in via email/password; server validates bcrypt hash & issues JWT.
UC7: Event CRUD	Admin creates, updates, deletes events or adjusts seating capacity limits.
UC8: Gate Validation	Admin scans or pastes Booking ID to mark admission pass as "Used".

9. Data Flow Diagram (DFD)

Level 0 DFD — Context Diagram:

The Context Diagram models EventTicket as a unified black-box process illustrating primary external entities and bidirectional information pipelines.



Level 1 DFD — Functional Process Decomposition:

Process Identifier	Operational Functionality
P1: Input Validation	Validates attendee email format, ticket count > 0, and non-empty name.
P2: Capacity Verifier	Inspects if event.availableSeats >= seatsBooked to guarantee availability.
P3: Seat Deduction Engine	Atomically decrements availableSeats and updates event timestamp in DB.
P4: QR Pass Synthesizer	Generates unique TKT ID and encrypts verification JSON into SVG QR code.
P5: WebSocket Broadcaster	Emits `seat-updated` payload to event room and `new-booking` to admin.
P6: JWT Auth & Verifier	Authenticates admin credentials via bcrypt and signs/verifies JWT tokens.
P7: Gate Check-in Handler	Matches scanned Booking ID and updates ticket status from Booked to Used.

10. Entity Relationship Diagram (ERD)

The logical relational data model of EventTicket is shown below. An Event entity maintains a one-to-many relationship with Bookings. Each Booking references an Event and attendee information with embedded QR verification data.

																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																	</
--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

11. What Contribution Would the Project Make?

For Students and Technical Learners:

Provides a pristine, real-world reference implementation demonstrating how to build full-stack systems using React 19, TypeScript, and Express. It demystifies real-time WebSockets (Socket.io), atomic database transactions, stateless JWT security, and vector QR code synthesis.

For Campus Organizers and Academic Communities:

Delivers an immediately deployable, zero-cost ticketing portal for college fests, symposiums, and coding competitions without paying commercial platform fees or configuring credit card merchant gateways.

Technical & Architectural Contribution:

Demonstrates modern software engineering principles including server-authoritative state synchrony, race-condition mitigation under concurrent bursts, print-ready digital passes, and modular component architecture.

Academic Contribution:

Exemplifies core curriculum competencies in Web Technology, Computer Networks (WebSockets), Operating Systems (Concurrency & Locking), Database Systems (ACID Transactions), and Information Security (JWT & bcrypt).

REFERENCES

- [1] Socket.io Documentation, "Bidirectional and low-latency communication for every platform," Socket.io, 2024. [Online]. Available: <https://socket.io/docs/v4/>
- [2] RFC 7519, "JSON Web Token (JWT)," Internet Engineering Task Force (IETF), May 2015. [Online]. Available: <https://tools.ietf.org/html/rfc7519>
- [3] React Documentation, "React 19 Architecture and Server Components," ReactJS.org, 2024. [Online]. Available: <https://react.dev>
- [4] Express.js, "Fast, unopinionated, minimalist web framework for Node.js," Expressjs.com, 2024. [Online]. Available: <https://expressjs.com>
- [5] Tailwind Labs, "Tailwind CSS v4 Documentation," Tailwindcss.com, 2024. [Online]. Available: <https://tailwindcss.com>
- [6] ISO/IEC 18004, "Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification," ISO, 2015.
- [7] N. Provos and D. Mazieres, "A future-adaptable password scheme (bcrypt)," in USENIX Annual Technical Conference, 1999.
- [8] M. Fowler, "Patterns of Enterprise Application Architecture," Addison-Wesley Professional, 2002.
- [9] Mozilla Developer Network, "WebSockets API," MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API